
MongoDB

- MongoDB is a document-oriented NoSQL database system that provides high scalability, flexibility, and performance. Unlike standard relational databases, which store data in tables consisting of rows and columns, MongoDB stores data in JSON-like structures referred to as documents. This makes it easy to operate with dynamic and unstructured data and MongoDB is an open-source and cross-platform database System.
- Founded in 2007
- Developer(s):MongoDB Inc.
  https://www.mongodb.com/

Terminology Differences between RDBMS & MongoDB

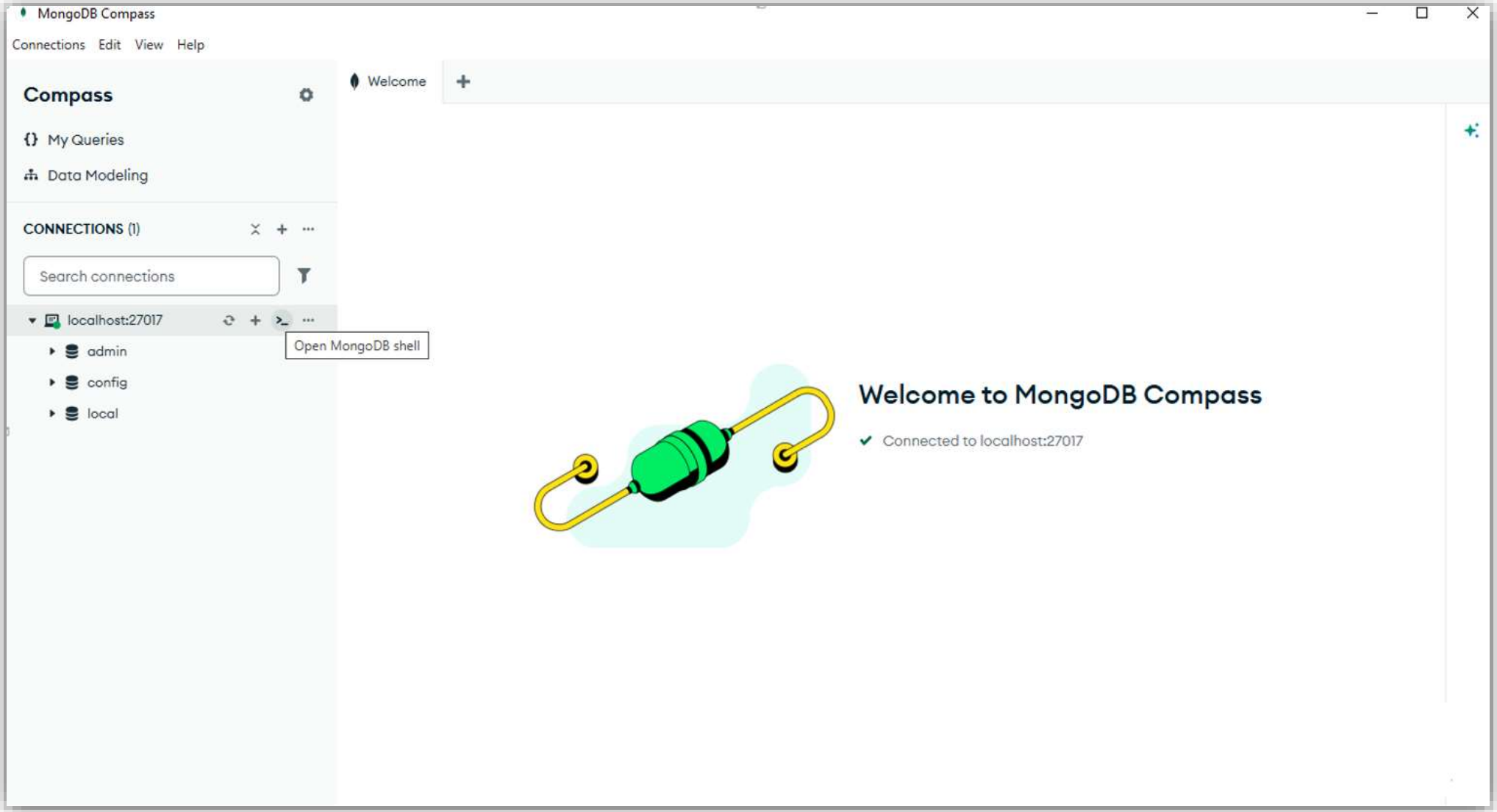
RDBMS	MongoDB
Database	Database
Table	Collection
Tuple/Row	Document
Column	Field

Document

- A document is a set of key-value pairs. Documents have dynamic schema. Dynamic schema means that documents in the same collection do not need to have the same set of fields or structure, and common fields in a collection's documents may hold different types of data.
- And the field values may include numbers, strings, booleans, arrays, or even nested documents.

MongoDB Compass & MongoDB Atlas

- MongoDB Atlas and MongoDB Compass are two tools provided by MongoDB.
- MongoDB Compass is a graphical(GUI) tool for managing MongoDB databases, enabling easy data exploration and querying in a visual environment.
- MongoDB Compass is integrated in the installation of MongoDB.
- MongoDB Atlas is a cloud-based, managed database service. It simplifies database hosting and maintenance.

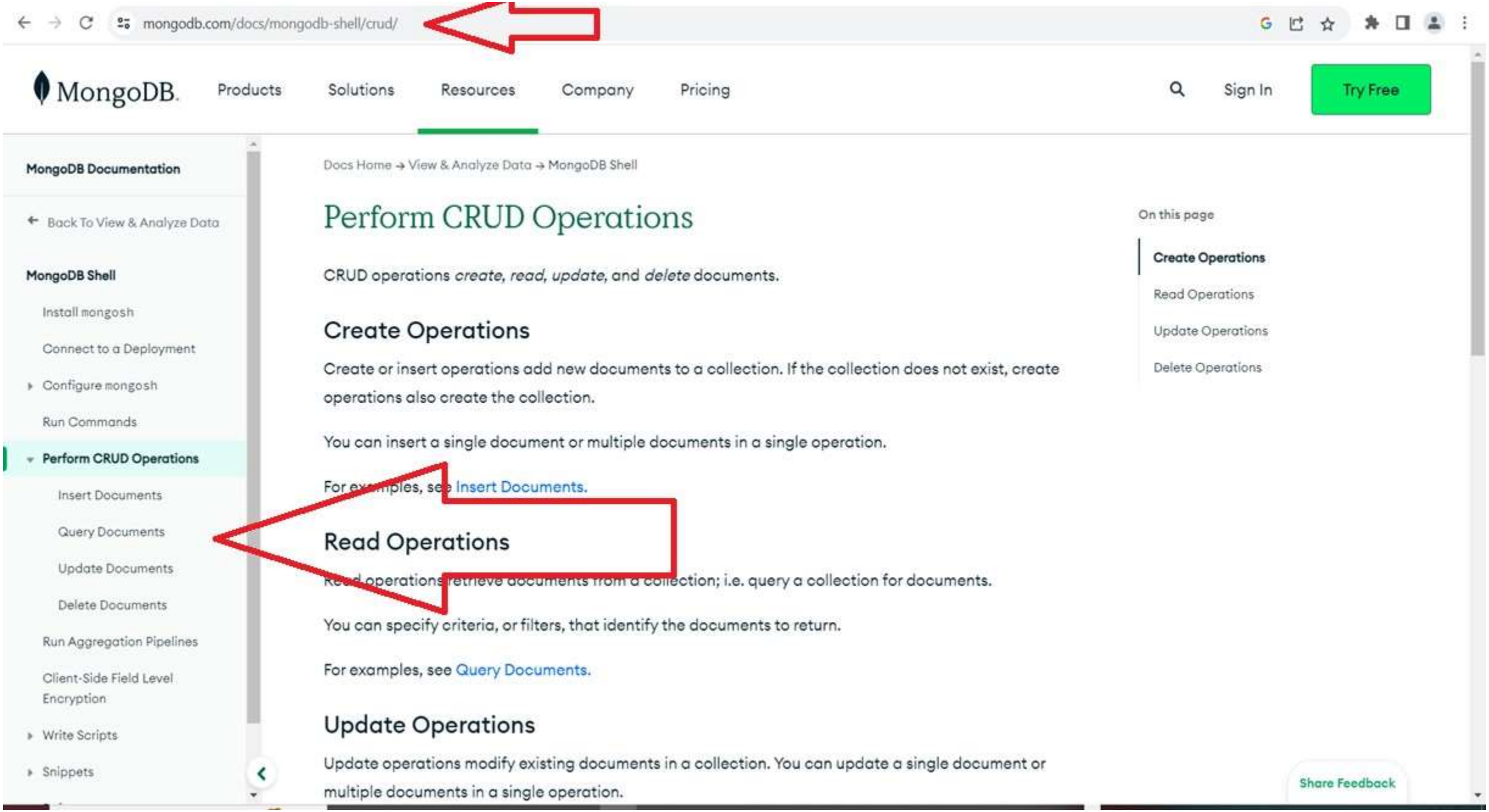

MongoDB Compass
Connections Edit View Help
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
Open MongoDB shell
admin
config
local
Welcome
Welcome to MongoDB Compass
Connected to localhost:27017

MongoDB main Commands & Functions

(commads/functions are case-sensitive)

- To Show Databases-- show dbs
- To check currently selected database– db
- Create database-- use DATABASE_NAME
- Switching database-- use DATABASE_NAME
- Drop database-- db.dropDatabase()

- Create collection-- db.createCollection(“COLLECTION_NAME”)
- List created collections-- show collections
- Drop collection-- db.collection_name.drop()


mongodb.com/docs/mongodb-shell/crud/
MongoDB.
Products
Solutions
Resources
Company
Pricing
Sign In
Try Free
MongoDB Documentation
Back To View & Analyze Data
MongoDB Shell
Install mongosh
Connect to a Deployment
Configure mongosh
Run Commands
Perform CRUD Operations
Insert Documents
Query Documents
Update Documents
Delete Documents
Run Aggregation Pipelines
Client-Side Field Level Encryption
Write Scripts
Snippets
Docs Home → View & Analyze Data → MongoDB Shell
Perform CRUD Operations
CRUD operations create, read, update, and delete documents.
Create Operations
Create or insert operations add new documents to a collection. If the collection does not exist, create operations also create the collection.
You can insert a single document or multiple documents in a single operation.
For examples, see Insert Documents.
Read Operations
Read operations retrieve documents from a collection; i.e. query a collection for documents.
You can specify criteria, or filters, that identify the documents to return.
For examples, see Query Documents.
Update Operations
Update operations modify existing documents in a collection. You can update a single document or multiple documents in a single operation.
On this page
Create Operations
Read Operations
Update Operations
Delete Operations
Share Feedback

- Insert document-- db. collection_name.insert(*<document or [document1, document2,…]>*)
- db. collection_name.insertOne()
- db. collection_name.insertMany()
- Read All Documents in a Collection-- db.collection.find()
- db.collection.find().pretty()
- db. collection_name.find({key:value})
- Update document—
- db. collection_name.update(SELECTION_CRITERIA, UPDATED_DATA)
- db.collection_name.updateOne(SELECTION_CRITERIA, UPDATED_DATA)

Query operators

RDBMS Where Clause Equivalents in MongoDB

To query the document on the basis of some condition, you can use following operations.

Operation	Syntax	Example	RDBMS Equivalent
Equality	{<key>:{$eg; <value>}}	db.mycol.find({"by":" Raghu' :"}).pretty()	where by = ':Raghu'
Less Than	{<key>:{$lt: <value>}}	db.mycol.find({"likes": {$lt:50}}).pretty()	where likes < 50
Less Than Equals	{<key>:{$lte: <value>}}	db.mycol.find({"likes": {$lte:50}}).pretty()	where likes <= 50
Greater Than	{<key>:{$gt: <value>}}	db.mycol.find({"likes": {$gt:50}}).pretty()	where likes > 50
Greater Than Equals	{<key>:{$gte: <value>}}	db.mycol.find({"likes": {$gte:50}}).pretty()	where likes >= 50
Not Equals	{<key>:{$ne: <value>}}	db.mycol.find({"likes": {$ne:50}}).pretty()	where likes != 50
Values in an array	{<key>:{$in: [<value1>, <value2>,..... <valueN>]}}	db.mycol.find({"name":{$in: ["Raj", "Ram", "Raghu"]}}).pretty()	Where name matches any of the value in :["Raj", "Ram", "Raghu"]

Logical Operators

- **AND in MongoDB--**

db. collection_name.find({ $and: [{<key1>:<value1>}, { <key2>:<value2>}]
})

- **OR in MongoDB--**

db.collection_name.find({$or: [{key1: value1}, {key2:value2}]}).pretty()

- **NOT – MongoDB--**

db.collection_name.find({$not: [{key1: value1}, {key2:value2}]}).pretty()

- Delete Document-- db.collection_name.remove(DELLETION_CRITTERIA)
- deleteOne, deleteMany, findOneAndDelete
- Remove All Documents-- db.collection_name.remove({})